Group 4 - CloudNine Systems

Milestone 1: Draft Analysis Report

Core design: 20 tables, four work packages of five tables. Updated October 15, 2026. Team names and acknowledgment remain pending.

1. Purpose and website scope

CloudNine supplies shared compute, storage and network services. The database connects client requests to actual resource assignments, measured usage and billed costs. The website is a read-only operations and billing dashboard, not a real cloud provisioning platform. Synthetic records and SQL queries demonstrate how the system supports decisions.

The central workflow is client -> reservation -> reservation_item -> resource_allocation, with deployment identifying the running workload. deployment_service connects a deployment to a service; usage_record measures consumption; charge records its cost; charge_share assigns portions to invoices, whose billing accounts identify the payers.

Two pages are sufficient. Operations shows capacity by location, reservation shortages, deployments and downtime. Billing shows estimated versus actual usage, charges, credits, payer allocations and invoices. No separate CRUD page is needed for each database table. Detailed page and query responsibilities are in Group4_Website_Plan.md.

2. Twenty-table organizational schema

A - Clients and reservations (5): client, user_account, reservation, reservation_item, billing_account. Clients may be individuals, teams or organizations. A client can have one parent. A user has at most one client affiliation in this simplified model; staff-only users can have no client. Reservations contain requested types, quantities, capacities and location constraints. A billing account identifies the responsible payer, who may differ from the consuming client.

B - Infrastructure (5): region, availability_zone, data_center, resource_type, resource. The location hierarchy is region -> availability_zone -> data_center -> resource. A resource has a type, tier and scalar capacity values. These tables supply location filters and grouped capacity statistics; they do not require separate website pages.

C - Deployments and usage (5): deployment, resource_allocation, resource_state, deployment_service, usage_record. Deployment identifies the workload associated with a reservation. Allocations record which resources serve it over time, including substitutions. State intervals record maintenance and failures. Provisioned services and estimated or actual usage support operations and billing.

D - Services and billing (5): service, service_rate, charge, charge_share, invoice. Services define billing units, and effective-dated rates define prices. Charges capture metered cost, direct fees or adjustments. Each share is an invoice line as well as a portion of a charge. Its payer is derived through invoice -> billing_account, avoiding a redundant payer field in charge_share.

3. Simplification and 3NF reasoning

The former 23-table design is reduced to 20. project is removed: deployments are named and grouped by client/reservation; reusable project metadata and project reporting are outside scope. client_membership is removed: user_account.clientId stores at most one current affiliation. Cross-client membership, membership roles and affiliation history are outside scope; client hierarchies still represent organizations and teams.

invoice_line is removed. charge_share now references invoice directly and no longer stores billingAccountId. A draft invoice is created for a payer before shares are allocated. A share is already the monetary line item, so another association table adds no independent information. Issued invoices retain their shares. This is a deliberate lifecycle change: there are no payer shares waiting outside an invoice.

The location hierarchy is retained because the scenario explicitly distinguishes regions, zones and data centers, and reservations may constrain any one of these levels. Merging all three into free-text resource fields would duplicate location information and weaken grouped capacity queries. The website presents them as filters and summaries rather than separate management screens.

1NF: capacities are scalar numbers, and requests, allocations, usage and shares occupy separate rows. Descriptive notes are not used to hide lists of resources or payers. 2NF: every table has a single-column primary key; non-key attributes depend on the whole identifier. The composite business identifiers in R9 have no stated partial dependencies.

3NF: resource stores dataCenterId only; region and zone are obtained through the location hierarchy. usage_record stores deploymentServiceId without repeating deployment, service or billing unit. service_rate stores a service and price interval without copying service descriptions. charge_share stores chargeId, invoiceId and allocatedAmount; its payer comes from the invoice. Invoice totals and resource occupancy are derived rather than stored.

A metered charge's amount is a posted monetary fact, and credits are separate adjustment rows. Its optional usageId is a conditional identifier for metered rows, not a candidate key for every charge. A charge selects exactly one context as described in R7. The model is intended to satisfy 3NF under these business dependencies; additional dependencies found during implementation must be reviewed.

4. Integrity rules and assumptions

R1 - Clients and responsibility. Every reservation belongs to one client. A client's optional parent cannot create cycles. Each user has zero or one current client; a client-facing user must have one, while staff-only users may have none. Staff actions reference active staff or dual-role accounts. Billing accounts can belong to a different client by agreement. No authentication implementation is required for the demonstration website.

R2 - Time and physical resources. Use UTC and half-open intervals [start, end); a nullable end means open-ended. End must exceed start. Each data center belongs to one zone, and each zone to one region. Resources are independently allocatable assets, not both a cluster and its components. Capacities and resource type are fixed once referenced; reconfiguration creates a new resource identifier. Resource-state intervals cannot overlap for the same resource.

R3 - Reservation fulfillment. An approved reservation has at least one item. Every allocation's deployment and item belong to the same reservation. Resources must match the item's type and minimum per-instance capacities after applying capacityFraction. At a selected time, planned or active assignments for an item cannot exceed its quantity. Full fulfillment requires that many distinct matching resources; otherwise the dashboard reports a shortage.

R4 - Capacity and substitutions. capacityFraction is greater than zero and at most one. Concurrent planned or active fractions total at most one per resource. Allocations must fit reservation dates, and active ones must fit deployment dates. Maintenance, failure and retirement block usable assignments. A replacement closes the old interval and records the preceding allocation, reason and assigning actor. Predecessor links cannot cycle and remain within the same deployment/item. Modified reservations must revalidate existing assignments; incompatible changes require a new reservation.

R5 - Location and quantities. A reservation item chooses at most one of preferredRegionId, preferredZoneId and preferredDataCenterId; ancestors are derived. locationMode distinguishes mandatory restrictions from preferences. Mandatory restrictions must be met; preference substitutions require a reason. Counts are positive;

capacities, usage and rates are nonnegative. Allocation fractions apply proportionally to all resource dimensions. Independent multidimensional partitioning is outside scope.

R6 - Services, usage and prices. Each provisioned service belongs to one deployment and one service offering. Resource-based usage requires an allocation in the same deployment covering the metered interval. Usage must fit the service interval. Each service has one immutable billing unit. Rate intervals for a service cannot overlap. Consumption spanning rate boundaries is split. Estimates and actuals remain separate and are never summed as actual use. Correction rows retain their predecessor, source, interval and record type; correction links cannot cycle or branch, and reports use the latest applicable row.

R7 - Charges and credits. Each charge fills exactly one of usageId, deploymentServiceId and adjustmentOfChargeId. Metered charges use the matching serviceRateId and normally equal quantity times unitPrice, rounded to cents. Direct fees and adjustments leave serviceRateId empty. A usage record supports at most one charge. Only actual usage supports final metered charges; estimates support provisional charges that are voided when actuals are charged. Final charges are immutable. Correct them with an authorized reversing adjustment and replacement charge. Credits are negative; manual adjustments require an actor and reason. Adjustment links cannot cycle.

R8 - Draft invoices and sharing. Create a draft invoice for each payer before allocating a charge. Each charge_share belongs to exactly one charge and one invoice; the responsible account is obtained from the invoice. A charge may be split across multiple payers' draft invoices. Finalized allocations total the charge; positive charges have nonnegative shares and credits have nonpositive shares. Before any share of a final charge is issued, all its shares must be finalized. Draft shares may be corrected, but invoiceId, billingAccountId on their invoice, and monetary shares cannot change after issuance. Issued invoices have at least one share. Invoice totals are calculated from shares. Store money in USD with fixed decimal precision. Payment collection and settlement are outside scope.

R9 - Business identifiers and enforcement. Additional uniqueness rules are user_account.email; region.name; resource_type.name; service.name; availability_zone(regionId, name); data_center(zoneId, name); resource(dataCenterId, localIdentifier); service_rate(serviceId, validFrom); charge_share(chargeId, invoiceId); charge.usageId when populated; and usage_record.supersedesUsageId when populated. No rule that a charge has only one share per payer is assumed: a payer may have different invoices. PK/FK are the only markers displayed on diagrams; uniqueness remains a database rule. Capacity totals, interval conflicts, minimum children, staff roles, ownership and share totals need transactional checks or triggers in addition to local constraints.

5. Scope and scenario coverage

Retained: individuals/teams/organizations, client contact and hierarchy, user versus payer separation, three location levels, resource types and capacities, availability intervals, advance requests, delayed allocation, deployment status and priority, scaling allocations, substitutions, service pricing, estimated/actual use, credits, cost sharing and invoices.

Simplified: one current client affiliation per user; deployments grouped by client/reservation instead of projects; payer shares created directly on draft invoices. These choices remove three tables and reduce joins without introducing unrelated tables to reach the 20-table target.

Deferred with explanation: component clusters, resource dependencies and graphical topology; full configuration lineage; reservation revisions and complete deployment/priority/access event logs; credentials, access-control enforcement and confidential views; contracts, deposits, qualification scoring, payments and cash refunds; time-of-day, volume and bundle pricing; automated cost-sharing rules, autoscaling decisions and allocation optimization. Current confidentiality, priority, scaling mode and staff actions remain descriptive metadata. Standard rates and authorized manual credits/fees cover the demonstration. The website reports stored decisions and synthetic activity; it does not operate real infrastructure.

Allocation, availability, prices and consumption correction history remain because they directly support the planned queries. Every omitted scenario detail above is an explicit limitation, subject to instructor feedback, rather than a claim of implemented coverage.

6. Example, data and implementation plan

A client reserves two compute instances requiring 2 vCPUs and 4 GiB each in a chosen region. Two matching 4-vCPU/8-GiB resources allocated at 0.5 each fulfill the request. A failure closes one assignment and creates a suitable replacement, preserving the reason. Actual compute consumption produces a USD 60 charge. Create draft invoices for two billing accounts, allocate USD 36 and USD 24 as charge_share rows, then issue both invoices after validation. Their totals come from the shares; a later authorized credit becomes a separate negative charge and share.

Populate at least one quarter of synthetic activity across multiple regions, zones, data centers, clients, resource types and services. Include maintenance, shortages, substitutions, estimates, actuals, credits and split payers. Eight aggregate/join queries and two read-only pages are specified in Group4_Website_Plan.md. Live monitoring, deployment provisioning and user login are not required. Meeting the 20-table threshold alone does not guarantee a grade; correct normalization, realistic records, meaningful queries and interpretation remain necessary.

Four members each own five tables and an end-to-end portion of the implementation. Responsibilities and review duties are in the Team Statement. Names, acceptance and shared repository access remain pending. This milestone contains the design and plan; implementation and query execution are later work.

Source basis and presentation

The course Group Project - DB.pdf and Group Project_ CloudNine Systems.pdf define the project and scenario. Assignment 3 and GChau's example guide singular lower_case table names, camelCase attributes, PK/FK markers, labelled cardinalities and a clear table-and-relationship layout. The latest visual reference guides rounded white table boxes, three divided columns, orthogonal lines and crow-foot cardinalities. The four diagrams repeat external primary-key reference boxes for readability. Previous designs remain in Git history.